

MILESTONE 1 REPORT: DATA ACQUISITION

Môn học: SEG301 - Search Engines & Information Retrieval

Nhóm: Birds Search Engine

Chủ đề: Topic 4 - Social Listening (Lắng nghe Mạng xã hội)

Nguồn dữ liệu chính: VOZ Forum (F17/F33)

1. TỔNG QUAN DỰ ÁN (Project Overview)

1.1 Kết quả đạt được

Dựa trên quá trình thu thập và xử lý dữ liệu, nhóm đã hoàn thành vượt mức các chỉ tiêu đề ra cho Milestone 1. Số liệu dưới đây được trích xuất từ file thống kê mới nhất (statistics_report.md):

Chỉ số	Giá trị đạt được	Yêu cầu	Trạng thái
Tổng số Documents	1,135,076	1,000,000	Vượt 13.5%
Tổng số từ (Tokens)	154,972,519	-	-
Độ dài trung bình	136.53 từ/doc	> 50 từ	Đạt chuẩn
Số lượng Thread	112,528	-	Đa dạng chủ đề
Số lượng Tác giả	31,063	-	Đa dạng quan điểm

1.2 Định nghĩa Document

Để phù hợp với bài toán **Social Listening**, nhóm định nghĩa một Document hợp lệ là:

- Một bài viết (Post) hoặc Bình luận (Comment) thảo luận về các vấn đề xã hội/công nghệ.

2. **Độ dài tối thiểu:** 50 từ (sau khi đã tách từ và loại bỏ stopword cơ bản).
3. **Ngôn ngữ:** Tiếng Việt (có hỗ trợ từ khóa tiếng Anh chuyên ngành).

2. QUY TRÌNH KỸ THUẬT (Technical Approach)

2.1 Kiến trúc Crawler

Nhóm chuyển đổi từ kiến trúc AsyncIO sang mô hình Multi-threading Worker Pool để đảm bảo khả năng vượt qua cơ chế chặn bot (Cloudflare) của VOZ, đồng thời duy trì hiệu suất cao khi xử lý hàng triệu requests.

a. Công nghệ cốt lõi (Core Stack):

- Engine: Python 3.x với `concurrent.futures.ThreadPoolExecutor`.
- Request Client: `cloudscraper` (thay vì `requests` thường) để giả lập browser fingerprint, tự động bypass các trang "Just a moment..." của Cloudflare.
- HTML Parser: `BeautifulSoup4` với parser `lxml` cho tốc độ phân tích cú pháp cực nhanh.
- Progress Tracking: `tqdm` để hiển thị tiến độ thời gian thực.

b. Cơ chế hoạt động (Concurrency Model): Hệ thống hoạt động theo mô hình Producer-Consumer với Thread Pool:

- Scout Thread (Producer): Quét các trang danh mục (F17, F33, F34...) để thu thập danh sách Thread ID.
- Worker Threads (Consumers): Mặc định 20 workers hoạt động song song. Mỗi worker nhận một Thread, crawl toàn bộ các page trong thread đó, parse comment và trích xuất dữ liệu.
- Connection Pooling: Sử dụng `requests.adapters.HTTPAdapter` với `pool_connections=10`, `pool_maxsize=20` để tái sử dụng kết nối TCP, giảm độ trễ bắt tay SSL.

c. Tính năng Tin cậy & Ổn định (Reliability Features):

- Cơ chế Checkpoint (Resume Capability):
 - Sử dụng class `CrawlCheckpoint` để tuần tự hóa (serialize) trạng thái vào file `crawl_checkpoint.pkl`.
 - Lưu trữ: Tập `crawled_threads`, `crawled_posts`, và trang cuối cùng đang crawl của từng forum.
 - Cho phép dừng và chạy lại crawler bất cứ lúc nào mà không bị mất dữ liệu hay crawl lại từ đầu.
- Anti-Blocking Strategy:
 - Random Delay: `time.sleep(random.uniform(0.1, 0.3))` giữa các request.
 - Auto-Retry: Tự động thử lại 3 lần nếu gặp lỗi mạng hoặc Timeout.
 - Xử lý mã lỗi 429 (Too Many Requests): Tự động ngủ 30 giây khi server phản hồi quá tải.

d. Tối ưu hóa Hiệu năng (Performance Optimization):

- Thread-Safe Writing: Sử dụng `threading.Lock()` để quản lý việc ghi file JSONL, cho phép nhiều luồng cùng ghi dữ liệu (Streaming Write) mà không gây lỗi race condition.
- Deferred Tokenization: Để tối đa hóa tốc độ thu thập, quá trình tách từ (Tokenization) được tách khỏi crawler và thực hiện ở giai đoạn hậu xử lý (Post-processing). Crawler chỉ

thực hiện làm sạch cơ bản (Clean HTML, Unicode NFC).

2.2 Pipeline Xử lý Dữ liệu (Data Processing)

Chúng tôi thiết kế một quy trình xử lý dòng (Streaming Pipeline) để đảm bảo hiệu năng khi làm việc với tập dữ liệu lớn (>1GB). Quy trình bao gồm 6 bước tuần tự:

1. **Unicode Normalization:** Chuẩn hóa bảng mã.
2. **Noise Removal:** Loại bỏ rác kỹ thuật (HTML, URL).
3. **Text Standardization:** Xử lý dấu câu và tiếng Anh.
4. **Tokenization:** Tách từ tiếng Việt.
5. **Filtering:** Lọc theo độ dài (Min Word Count).
6. **De-duplication:** Loại bỏ dữ liệu trùng lặp.

2.3. Chi tiết Kỹ thuật

Chuẩn hóa Unicode (Normalization)

- **Vấn đề:** Tiếng Việt trên máy tính có 2 dạng tổ hợp (Composite) và dựng sẵn (Precomposed). Ví dụ: chữ "á" có thể được lưu là a + dấu sắc hoặc ký tự á riêng biệt. Điều này khiến máy tính hiểu nhầm là 2 từ khác nhau.
- **Giải pháp:** Sử dụng thư viện unicodedata để chuyển toàn bộ văn bản về chuẩn **NFC (Normalization Form C)**.
- **Tác dụng:** Đảm bảo nhất quán về mặt hiển thị và tìm kiếm cho tiếng Việt.

Loại bỏ nhiễu (Noise Removal)

- **Vấn đề:** Dữ liệu crawl từ web chứa các thẻ HTML (<div>,
, <script>), đường dẫn URL, email và icon rác.
- **Giải pháp:** Sử dụng **Regular Expression (Regex)** để nhận diện và loại bỏ:
 - Xóa toàn bộ thẻ script/style: `r'<(script|style).*?>.*?</1>'`
 - Xóa thẻ HTML: `r'<.*?>'`
 - Xóa URL/Email: `r'http\S+|mailto:\S+|@|\.`
- **Kết quả:** Chỉ giữ lại nội dung văn bản thuần túy (Plain Text).

Xử lý Tiếng Anh và Dấu câu (English Handling)

1. **Thách thức:** Trong các diễn đàn công nghệ (Voz/TinhTe), người dùng thường viết trộn lẫn tiếng Anh và tiếng Việt (Code-switching). Các bộ tách từ tiếng Việt thường xử lý sai khi từ tiếng Anh dính liền dấu câu (Ví dụ: "Python," bị hiểu là một từ lạ).
2. **Giải pháp:** Áp dụng kỹ thuật **Regex Padding**:
 - Thay thế tất cả các ký tự **KHÔNG PHẢI** là: *Chữ cái (Việt + Anh), Số, Khoảng trắng* bằng một khoảng trắng.
 - Regex: `r'[^\w\sàáạ...]'`
3. **Ví dụ:**

- Input: "Học Python,Java và C++."
 - Output: "học python java và c "
4. **Tác dụng:** Giúp tách biệt hoàn toàn các từ khóa tiếng Anh quan trọng khỏi dấu câu, giúp bộ tách từ hoạt động chính xác hơn.

Tách từ Tiếng Việt (Vietnamese Tokenization)

1. **Công cụ:** Thư viện **PyVi** (Python Vietnamese Core).
2. **Cơ chế:** Tiếng Việt là ngôn ngữ đơn lập, ý nghĩa được tạo thành từ việc ghép các từ đơn. Khoảng trắng không phải là dấu hiệu tách từ duy nhất (khác với tiếng Anh).
3. **Hoạt động:** PyVi sử dụng thuật toán thống kê để nối các từ ghép bằng dấu gạch dưới `_`.
 - Input: "sinh viên đại học fpt"
 - Output: "sinh_viên đại_học fpt"
4. **Ý nghĩa:** Biến cụm từ có nghĩa thành một token duy nhất (Term), giúp thuật toán tìm kiếm (như TF-IDF/BM25) hiểu được ngữ cảnh chính xác thay vì tìm rời rạc từng từ "sinh", "viên".

Bộ lọc Chất lượng (Quality Filter)

- **Tiêu chí:** Chỉ giữ lại các tài liệu có độ dài **≥ 50 từ** (sau khi tách từ).
- **Lý do:** Các comment ngắn (như "hóng", "bóc tem", "hay quá") chiếm số lượng lớn nhưng không mang lại giá trị thông tin (Information Value) cho Search Engine.
- **Thực thi:** Đếm số lượng token sau khi tách từ. Nếu `word_count < 50`, hệ thống sẽ loại bỏ tài liệu đó khỏi tập dữ liệu sạch.

Khử trùng lặp (De-duplication)

- **Vấn đề:** Một bài viết có thể bị crawl nhiều lần, hoặc nội dung spam xuất hiện lặp lại ở nhiều nơi.
- **Giải pháp:** Sử dụng thuật toán **Hashing (MD5)**.
 - Tạo một mã băm (Fingerprint) cho nội dung văn bản gốc của mỗi tài liệu.
 - Lưu các mã băm đã gặp vào một HashSet (cấu trúc dữ liệu tra cứu $O(1)$).
 - Nếu mã băm của tài liệu mới đã tồn tại trong Set -> Loại bỏ.
- **Hiệu quả:** Loại bỏ triệt để các bản sao chính xác (Exact Duplicates) mà không tốn nhiều bộ nhớ RAM.

3. PHÂN TÍCH DỮ LIỆU (Data Insights)

3.1 Thống kê tổng quan

Tổng số documents	1,135,076
-------------------	-----------

Tổng số từ (Tokens)	154,972,519
Số lượng thread	112,528
Số lượng tác giả	31,063

3.2 Thống kê độ dài của từ vựng

Ngắn nhất	50 từ
Dài nhất	8,213 từ

3.3 Phân bố độ dài Document

Dữ liệu tập trung chủ yếu vào các bình luận có độ dài trung bình (50-200 từ), phù hợp với tính chất thảo luận trên diễn đàn.

Khoảng độ dài	Số lượng	Tỷ lệ phần trăm
50 - 100 từ	646,307	56.94%
101 - 200 từ	336,139	29.61%
201 - 500 từ	120,397	10.61%
501 - 1000 từ	25,780	2.27%
> 1000 từ	6,453	0.57%

4. LƯU TRỮ & CẤU TRÚC (Storage & Structure)

4.1 Định dạng lưu trữ

Dữ liệu được lưu dưới dạng **JSON Lines (.jsonl)**.

- **Ưu điểm:** Tối ưu cho Big Data, có thể đọc/ghi theo dòng (Streaming) mà không cần load toàn bộ 2GB dữ liệu vào RAM.

4.2 Schema dữ liệu

Mỗi dòng trong file là một object JSON với cấu trúc:

JSON

```
{
  "doc_id": "voz_t1055393_p2",
  "thread_id": "t1055393",
  "content": "Nội dung gốc chưa xử lý...",
  "text_clean": "nội_dung đã xử lý tách từ...",
  "author": "username",
  "source": "voz",
  "url": "https://voz.vn/t/...",
  "word_count": 125
}
```